

D-Bash — Handoff technique

Récapitulatif complet des décisions, algorithmes et architecture implémentés

Ce document résume tout ce qui a été conçu et implémenté depuis le début du projet, en partant du code de base que tu avais fourni (proto-nicolas/server.cpp initial + client.cpp) jusqu'à l'état actuel. Il t'explique les décisions prises, les algorithmes choisis, l'architecture complète, et ce qui reste à faire.

1. Point de départ — ce que tu avais fait

Ton server.cpp initial contenait :

- ROUTER ZeroMQ qui écoute sur le port de ce nœud
- Thread heartbeat qui envoie la charge toutes les 3 secondes via DEALER sockets
- Détection de nœuds morts : si pas de heartbeat depuis 10s → Liveness::Dead
- Stubs vides pour ClientCommand et ClientHandshake
- client.cpp : squelette DEALER avec handshake et boucle interactive

Deux problèmes de conception ont été identifiés et corrigés :

1. Le heartbeat_thread avait un bug de logique — il envoyait uniquement aux nœuds Alive mais ne gérât pas Uninitialized. Corrigé pour envoyer aussi aux Uninitialized (première prise de contact).
2. Le receive loop utilisait un continue après le bloc pair_pull, ce qui bloquait le traitement du router quand le watchdog notifiât. Les deux blocs sont maintenant indépendants.

2. Architecture globale

Le projet est découpé en deux binaires. Le serveur (proto-nicolas/server.cpp) tourne en démon sur chaque nœud et gère toute la logique distribuée. Le shell (src/dbash) est l'interface utilisateur — il se connecte au serveur local via ZeroMQ et lui délègue tout.

```
[Utilisateur tape "ls -la"]
|
[dbash - repl.cpp]
-> zmq_send(ClientCommand, "ls -la")
|
[server.cpp - noeud local]
-> seuil dynamique : ma_charge > moyenne_cluster * 1.5 ?
  -> OUI : jitter 0-300ms -> recalcul -> RemoteCommand vers noeud cible
  -> NON : fork/exec local + broadcast_load() immediat
-> repondre "global_pid:XXX
<output>" au client
|
[dbash] add_job(global_pid) + afficher output
```

Fichiers partagés entre les deux binaires :

cluster_state.hpp — contient PeerInfo, Liveness, et les déclarations extern des variables globales (peers, peers_mutex, my_node_id, num_nodes). Ces variables sont définies dans server.cpp mais accessibles depuis load_balancer.cpp.

message_types.hpp — enum de tous les types de messages ZeroMQ échangés entre nœuds et clients.

3. Protocole ZeroMQ — tous les types de messages

Chaque message ZeroMQ est composé de deux frames : un byte de type (MessageType) et un payload string. Le pattern ROUTER/DEALER est utilisé partout.

```
enum class MessageType : uint8_t {
    Heartbeat           = 1, // noeud->noeud : broadcast charge
    ClientHandshake     = 2, // client->serveur : connexion initiale
    ClientAcknowledgement = 3, // serveur->client : OK pour handshake
    ClientCommand       = 4, // client->serveur : lancer une commande
    ClientReturnValue    = 5, // serveur->client : resultat + global_pid
    RemoteCommand       = 6, // noeud->noeud : deleguer une commande
    RemoteReturnValue    = 7, // noeud->noeud : resultat de la commande
    RemoteKill          = 8, // client->serveur->noeud : kill distant
    JobFinished         = 9, // serveur->client : job background termine
    RemoteJobFinished   = 10 // noeud->noeud : job bg distant termine
};
```

Format des payloads importants :

- ClientReturnValue foreground : **"global_pid:XXX\n<output>"**
- ClientReturnValue background : **"global_pid:XXX"**
- JobFinished : **"global_pid:XXX"**
- RemoteKill : **"target_node:sig:local_pid"**

4. Load Balancer — algorithmes et décisions

`select_target(node_id, peers)`

Retourne l'indice du nœud alive le moins chargé. Exclut `node_id` lui-même. Fallback sur `node_id` si aucun pair n'est alive. Décision : on n'inclut pas soi-même dans la comparaison car le seuil dynamique a déjà décidé qu'on est surchargé avant d'appeler cette fonction.

`get_global_load(peers)`

Moyenne des charges de tous les nœuds alive. Retourne 0.0f si personne n'est alive — dans ce cas le seuil ne se déclenche pas et on exécute localement par défaut.

Seuil dynamique — décision de redirection

La logique est dans le case `ClientCommand` de `server.cpp`. On compare sa charge locale à la moyenne du cluster multipliée par 1.5. Ce facteur 1.5 signifie qu'on ne redirige que si on est 50% plus chargé que la moyenne — évite les redirections inutiles quand tout le monde est au même niveau.

```
float my_load      = stof(getLoadAverage());
float global_charge = get_global_load(peers); // avec lock
int  target        = select_target(my_node_id, peers);

if (my_load > global_charge * 1.5 && target != my_node_id) {
    // Jitter avant de rediriger
    sleep(random(0, 300ms));
    // Recalculer apres le delai
    global_charge = get_global_load(peers);
    target        = select_target(my_node_id, peers);
    if (target != my_node_id) { /* rediriger */ }
}
// sinon fork local
```

Thundering herd — pourquoi le jitter ?

Problème : si 4 nœuds reçoivent une commande simultanément et voient tous le même nœud comme le moins chargé, ils l'inondent tous en même temps. Solution en deux parties :

1. Jitter (0-300ms) avant la redirection : désynchronise les décisions. La valeur 300ms est cohérente avec l'intervalle heartbeat de 3s — assez pour que les charges se mettent à jour entre les décisions.
2. Recalcul après le jitter : après le délai, on recalcule la cible avec les charges fraîches — si entre-temps le nœud cible a reçu d'autres jobs, on pourrait choisir quelqu'un d'autre.

Heartbeat réactif : après chaque fork, `broadcast_load()` envoie immédiatement la nouvelle charge à tous les pairs alive sans attendre les 3 secondes. Cela réduit la fenêtre pendant laquelle les autres nœuds voient une charge obsolète.

5. PID global — encodage unique sur le réseau

Chaque processus sur le cluster a un identifiant unique sur 32 bits. Les 16 bits de poids fort contiennent le `node_id`, les 16 bits de poids faible contiennent le PID Unix local.

```
uint32_t global_pid = (node_id << 16) | local_pid;

// Decodage :
int node_id  = global_pid >> 16;
int local_pid = global_pid & 0xFFFF;
```

Cette convention garantit l'unicité même si deux machines ont des processus avec le même PID local. Elle est décodable sans table centralisée.

6. Redirection nœud->nœud — foreground et background

Quand node 0 redirige vers node 1, il doit mémoriser quel client attend la réponse. Deux maps distinctes selon foreground/background :

pending_redirections — foreground

Map target_node_id -> client_sender_id. Effacée quand RemoteReturnValue arrive. Flux : node 0 envoie RemoteCommand -> node 1 exécute -> node 1 répond RemoteReturnValue via peers[0].dealer -> node 0 retrouve le client et lui envoie ClientReturnValue.

pending_bg_redirections — background

Map target_node_id -> client_sender_id. NON effacée à RemoteReturnValue — on attend encore RemoteJobFinished. Flux complet :

```
node 0 -> RemoteCommand (bg) -> node 1
node 1 -> fork + enregistre dans background_jobs
node 1 -> RemoteReturnValue (global_pid) -> node 0
node 0 -> ClientReturnValue (global_pid) -> client [pending_bg_redirections NON effacée]
...job se termine...
node 1 watchdog -> RemoteJobFinished -> node 0
node 0 -> JobFinished -> client [pending_bg_redirections effacée]
```

Pourquoi peers[source_node].dealer et pas router.send(sender) ?

Quand node 1 reçoit un RemoteCommand de node 0, le sender frame contient l'identité du dealer de node 0 (= '0'). Si node 1 répond avec router.send(sender), il envoie vers l'identité '0' sur son propre router — mais node 0 écoute sur son propre router, pas sur celui de node 1. Il faut donc que node 1 réponde via peers[source_node].dealer->send() pour que le message arrive sur le router de node 0.

7. Watchdog thread — notification de fin de job background

Un seul thread de surveillance pour tous les processus background. Il fait waitpid(-1, ...) en boucle — attend n'importe quel enfant. Quand un processus se termine, il notifie le receive loop via socket PAIR inproc.

Pourquoi socket PAIR inproc ?

ZeroMQ n'est pas thread-safe — on ne peut pas envoyer sur le router depuis un thread séparé. La socket PAIR inproc://jobdone permet au watchdog d'envoyer un message au thread principal qui lui transmet ensuite au client via le router. C'est le seul moyen propre de faire communiquer deux threads ZeroMQ.

Format de notification :

```
// Watchdog -> receive loop
"global_pid|sender_id|is_remote"
// is_remote=0 : job local, sender_id = client_id
// is_remote=1 : job distant, sender_id = node source (ex: "0")
```

zmq_poll dans la receive loop :

Le receive loop poll maintenant deux sockets simultanément : le router (messages réseau) et pair_pull (notifications watchdog). Les deux blocs if sont indépendants — pas de continue entre les deux. Ceci est critique : un continue après le bloc watchdog empêcherait le traitement des messages réseau.

8. REPL côté client — repl.cpp

Connexion au démarrage

repl_run(node_id) charge config_servers.txt, crée un socket DEALER avec l'identité 'client_N', se connecte à l'endpoint[node_id], et fait le handshake ClientHandshake/ClientAcknowledgement avant de lancer la boucle readline.

execute_command — boucle de réception

Après avoir envoyé ClientCommand, le REPL attend la réponse dans une boucle while. Si un JobFinished arrive en chemin (job background terminé pendant qu'on attend la réponse), il est traité immédiatement et la boucle continue d'attendre la vraie réponse.

```
while (true) {
    auto [t, p] = zmq_recv();
    if (t == MessageType::JobFinished) {
        handle_job_finished(p); // marquer finished dans job_table
        continue;              // continuer d'attendre ClientReturnValue
    }
    type = t; payload = p;
    break;
}
```

check_job_notifications()

Poll non-bloquant (10ms) appelé dans ps, wait, et après chaque commande. Vide les JobFinished qui seraient arrivés pendant que l'utilisateur tapait (entre deux commandes, personne n'écoutait le socket).

Parsing de la réponse

Le serveur répond toujours avec global_pid en tête. Pour le foreground : 'global_pid:XXX\n<output>'. Pour le background : 'global_pid:XXX'. Le REPL parse le newline pour séparer pid et output, ajoute le job dans job_table avec finished=!background.

Built-ins

ps : appelle check_job_notifications() puis affiche job_table. wait : boucle avec check_job_notifications() toutes les secondes jusqu'à ce que tous les jobs background soient finished. kill : si node_id == my_node_id → kill direct, sinon zmq_send(RemoteKill, 'node:sig:pid').

9. Job table — côté client

La job_table côté REPL contient tous les jobs lancés par cet utilisateur, peu importe sur quel nœud ils tournent. C'est la vue du client, pas du cluster. Pour un vrai ps global (tous les clients), il faudrait interroger les autres nœuds via un nouveau type de message — non implémenté.

```
struct Job {
    uint32_t global_pid; // (node_id << 16) | local_pid
    pid_t    local_pid;  // pour kill local direct
    int      node_id;    // pour savoir ou tourne le process
    string   command;
    bool     background;
    bool     finished;
};
```

10. Ce qui reste à implémenter

Tolérance aux fautes — PRIORITÉ (thème choisi)

La détection de panne est faite — un nœud passe Dead après 10s sans heartbeat. Ce qui manque : quand un nœud passe Dead, parcourir la job_table du client et relancer les jobs non-finished qui tournaient sur ce nœud. Le problème est que le client (REPL) ne sait pas qu'un nœud est tombé — c'est le serveur local qui a cette information. Il faut un nouveau message NodeDead que le serveur envoie au client connecté quand il détecte la mort d'un pair.

```
// Dans heartbeat_thread, quand peers[j] passe Dead :
// Envoyer NodeDead au client connecte
// MessageType::NodeDead = 11, payload = "node_id:J"

// Dans repl.cpp, handle_node_dead(node_id) :
// for (Job& j : job_table) {
//     if (j.node_id == dead_node && !j.finished) {
//         // relancer : zmq_send(ClientCommand, j.command)
//     }
// }
```

Démarrage sans argument — détection automatique du node_id

Actuellement './dbash 0' requiert de passer le node_id. Le sujet attend un shell transparent pour l'utilisateur. Solution : au démarrage, lire l'IP locale, la comparer avec config_servers.txt pour déduire automatiquement le node_id.

Nettoyage avant démo

- Retirer tous les [DEBUG check] dans repl.cpp
- Retirer le [DEBUG] RemoteReturnValue dans server.cpp
- Vérifier que my_load = stof(getLoadAverage()) et non 999.0f

Secondaire

- ps -l format long avec stats CPU/mémoire
- Interface pour modifier les machines participantes (mentionné dans le sujet)
- Seuil fixe en plus du dynamique

11. Compilation et lancement

```
# Compiler les deux binaires
make

# Lancer le cluster (4 terminaux)
./server 0
./server 1
./server 2
./server 3

# Lancer le shell (attendre ~10s que les heartbeats s'échangent)
./dbash 0

# Installer ZeroMQ si nécessaire
sudo apt install libzmq3-dev

# Compiler manuellement server
```



```
g++ -std=c++17 server.cpp src/load_balancer.cpp -o server -lzmq -pthread -I. -Isrc
```

```
# Compiler manuellement dbash
```

```
g++ -std=c++17 src/main.cpp src/repl.cpp src/parser.cpp src/job_table.cpp src/load_balancer.cpp -o dbash -lzmq
```